Name: Gaurika Mittal

Batch: 15

Sap ID: 590034892

Experiment-58

Algorithm

1. Start.
2. Read the size of the array size.
3. Declare a variable-length integer array arr[size].
4. Read size elements into the array.
5. Read the value of k.
6. Pass the address of arr, size, and k to the function kth_smallest().
7. In kth_smallest():
    - Check whether k is valid.
    - If k <= 0 or k > size, display -1 and return.
    - Use Bubble Sort to arrange the array in ascending order:
        - Compare each element with the next element.
        - If the current element is greater than the next element, swap them.
        - Repeat until the entire array is sorted.
    - The kth smallest element will be present at index k - 1.
    - Display the element at arr[k - 1].
8. Stop.

Test Case Table

Test Case	Size	Array Elements	k	Sorted Array	Expected Output
1	5	5 2 8 1 4	3	1 2 4 5 8	4
2	6	7 2 5 2 9 1	3	1 2 2 5 7 9	2
3	4	10 20 30 40	1	10 20 30 40	10
4	5	9 3 7 5 1	5	1 3 5 7 9	9
5	4	8 3 6 2	5	—	-1
6	4	8 3 6 2	0	—	-1

C Program

```
#include <stdio.h>


void kth_smallest(int *arr, int size, int k)
```

```
{
    int i, j, temp;

    // Check if k is valid
    if (k <= 0 || k > size)
    {
        printf("-1\n");
        return;
    }

    // Bubble Sort
    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }

    printf("Kth smallest element: %d\n", *(arr + k - 1));
}

int main()
{
    int size, k, i;

    printf("Enter size of array: ");
```

```
    scanf("%d", &size);

    int arr[size];

    printf("Enter %d elements:\n", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    kth_smallest(arr, size, k);

    return 0;
}
```